

CMSC411 Assignment 1

Fall 2024

Dues: On-time (September 13 by 11:00 PM) / Late (September 14 by 11:00 PM)

General instruction

- Submit a PDF file to GradeScope – You may edit the PDF file directly, scan your work on paper, or can also use an app (e.g., Scannable) to create a PDF file from photos of your work.
- After uploading your work, **don't forget to create mapping** between questions and your answers. 5 points will be deducted if mapping is not or incorrectly given.
- Unless specified, all questions will be graded.
- Unless specified, **justify your answers always**. If justifications are missing or do not show reasonably how your answers are derived, 0 points will be assigned regardless of the correctness of the answers.
- Write **neatly** in case you write your answers on paper. **Especially, for table completion questions, if your answers are not given neatly in well-drawn tables, the answers won't be graded.** It is also recommended to print this file and put your answers on the printout.
- **This assignment must be done by yourself without any unauthorized assistance.** Write your name and numeric student ID in the space below, and put your signature after reading the pledge statement.

Name: James Zhang

SID (Numeric): 118843940

I pledge my honor that all parts of this assignment were done by me individually and without any unauthorized assistance or collaboration with others.

Signature: 

Q1. Speedup

Assume that you are considering to enhance a machine by adding a new faster execution mode to it. When a computation is run in the fast mode, it is 10 times faster than using the normal mode of execution.

- (a) What percentage of the fast mode is needed to achieve a speedup of 2?

Speedup = (old time) / (new time) = $1 / [(1-f) + f/10] = 2$, and we have to solve this for f

$$2(1-f) + f/5 = 1$$

$$2 - 2f + f/5 = 1$$

$$-9f/5 = -1$$

$$f = 5/9$$

Therefore we need $f = 5/9$ which is around 55.6% needs to use fast mode to obtain a speedup of 2

- (b) What percentage of fast mode is required to achieve half of the maximum speedup? Note that the maximum speedup will be achieved when fast mode is used continuously.

If the max speedup is 10, then half the max speedup is 5

We need to solve $1 / [(1-f) + f/10] = 5$

$$5(1-f) + f/2 = 1$$

$$5 - 5f + f/2 = 1$$

$$-9f/2 = -4$$

$$f = 8/9$$

Therefore $f = 8/9$ which is around 88.9% of needs to use fast mode to obtain 1/2 of the maximum speedup

Q2. time command

Suppose that Unix *time* command tells that 90 seconds were spent for running user code, the wall-clock time was 3 mins, and the utilization of the CPU was 70%. How much time was spent for running kernel (system) code?

CPU Utilization (Percentage) = (user time + system time) / (wall clock time)
and we want to solve this for "system time." Plugging in known variables

$$0.7 = (90s + \text{system time}) / 180s$$

$$126s = 90s + \text{system time}$$

$$\text{system time} = 36s$$

Therefore, 36s was spent running kernel (system) code.

Q3. MIPS ISA

Consider the following fragment of C code that increases the value of every array element by 1:

```
for( ; i < 100 ; i++){
    a[i]++;
}
```

Consider the following assumptions and constraints to answer this question.

- Use MIPS32 instructions. For example, use ADD instead of DADD (for 64-bit). A few more MIPS32 instructions (just for your reference): ADDI, LW, SW, SLL, SRL, SLT, LUI, ORI, AND.Z
- Assume that (1) `i` is a 32-bit integer, which is stored in memory at the address `0x7FFF_FFFC` (occupying 4 bytes from `0x7FFF_FFFC` to `0x7FFF_FFFF`), and that (2) `0` is stored in the space. You will write code that loads the value to the register `$t0` once (before the loop code), and then uses the register during the loop (i.e., for `i++`) instead of accessing memory repeatedly. This approach will also help you simplify the code.
- Assume that the first array element is stored in memory at the address `0x7FFF_FE00` (occupying 4 bytes from `0x7FFF_FE00` to `0x7FFF_FF8F`).
- Do NOT use Load Immediate (`li`) instruction.
- Do NOT use Load Address (`la`) instruction.
- Do NOT use labels for `i` and `a`. Load the 32-bit memory addresses of the variables into register(s). Note that memory addresses are 32 bit wide but the immediate value field of ADDI is only 16 bit long. We discussed in class a method to load a 32-bit value into a register.

(a) Write the MIPS code.

```
addi $t1, $zero, 0x7FFF
sll $t1, $t1, 16
ori $t1, $t1, 0xFFFC
lw $t0, 0($t1)
addi $t2, $zero, 0x7FFF
sll $t2, $t2, 16
ori $t2, $t2, 0xFE00
addi $t3, $zero, 100
FOR
bge $t0, $t3, END
lw $t4, 0($t2)
addi $t4, $t4, 1
sw $t4, 0($t2)
addi $t0, $t0, 1
addi $t4, $t4, 4
j FOR
END:
```

- (b) What is the code size in bytes?

There are 14 instructions which are each 4 bytes and 1 pseudoinstruction which is 8 bytes, so the total code size is

$$14(4) + 1(8) = 56 + 8 = 64 \text{ bytes}$$

- (c) How many instructions are executed at run-time? If an instruction is executed multiple times, count all. No points will be given for incorrect code tracing.

There are 8 instructions before the for loop, so each of those runs once. There are $(100-i)$ iterations of the for loop, where i is the initial i value, and 7 instructions that will each run once per iteration. The pseudoinstruction bge will only run once. Therefore, the total number of instructions executed at run-time is $8 + 7(100-i) + 1$

$$= 709 - 7i \text{ instructions executed at run-time}$$

In the case $i = 0 \rightarrow 100$ iterations, 709 instructions are run at run-time

- (d) How many memory accesses (both reads and writes) are needed while running your code? No points will be given for incorrect code tracing.

There's one memory access before the for loop, and 2 for every iteration of the for loop. Therefore, there are

$$1 + 2(100-i) = 201 - 2i \text{ total memory accesses}$$

In the case $i = 0 \rightarrow 100$ iterations, 201 total memory accesses.

Q4. MIPS ISA

Consider the following fragment of C code that counts the number of 1s in the binary representation of an integer:

```
do{
    c += v % 2;
    v /= 2;
}while(v > 0);
```

Consider the following constraints to answer this question.

- Assume that `c` and `v` are 32-bit integers.
- Use MIPS32 instructions. For example, use `ADD` instead of `DADD` (for 64-bit). A few more MIPS32 instructions (just for your reference): `ADDI`, `LW`, `SW`, `SLL`, `SRL`, `SLT`, `LUI`, `ORI`, `AND`.
- Assume that all variables are kept in memory (at `0x7FFF0004` and `0x7FFF0000` for `c` and `v`, respectively). Also assume that `c` is initialized to 0 – i.e., zero is the value stored at the memory address `0x7FFF0004` (occupying 4 bytes from `0x7FFF0004` to `0x7FFF0007`).
- Do NOT use integer division instruction.
- Do NOT use Load Immediate (`li`) instruction.
- Do NOT use Load Address (`la`) instruction.
- Do NOT use labels for `c` and `v`. Load the memory addresses of the variables (given above) into register(s). Note that memory addresses are 32 bit wide but the immediate value field of `ADDI` is only 16 bit long.

- (a) Write the MIPS code. Do NOT give system call code block or data section, already given in the provided starter code. You may use the code for checking the correctness of your MIPS code by loading the code to *QtSpim* or *Mars* for execution.

```
# v
addi $t1, $zero, 0x7FFF
sll $t1, $t1, 16
lw $t2, 0($t1) # t2 = v
# c
addi $t3, $zero, 0x7FFF
sll $t3, $t3, 16
ori $t3, $t3 0x0004
lw $t4, 0($t3) # t4 = c

WHILE:
andi $t5, $t2, 0x0001
add $t4, $t4, $t5
srl $t2, $t2, 1
bgtz $t2, WHILE
sw $t2, 0($t1) # v
sw $t4, 0($t3) # c
```

- (b) What is the code size in bytes?

13 instructions so $13(4) = 52$ bytes

- (c) How many instructions are executed at run-time if v and c had 31 and 0, respectively? If an instruction is executed multiple times, count all. No points will be given for incorrect code tracing.

31 = 11111 in binary, so there will be 5 iterations of the while loop.

6 instructions before the while loop, 2 instructions after the while loop, and 5 iterations with 4 instructions per iteration so

$6 + 5(4) + 2 = 6 + 20 + 2 = 28$ instructions at run-time

- (d) Once again, if v and c had 31 and 0, how many memory accesses (both reads and writes) are needed while running your code? No points will be given for incorrect code tracing.

2 before the for loop and 2 after the for loop so

4 memory accesses total.